

The listing transaction

Short answer: the ordering is right — unlike buy, it waits for the receipt before telling the backend. But it has a rejection bug that pushes users into a doomed transaction, and a failure mode that can strand a domain listed on-chain with no record anywhere off it.

01 What actually happens

Two transactions, in `src/component/modals/DomainListingModal.tsx` — an ERC721 `approve`, then `createListing`.

1 Validate the form

Date, price, keywords, token ID. Fine, though each error returns one at a time rather than validating the whole form.

2 Chain check bails instead of switching

Same pattern as buy: sets the wrong-network flag and returns, so the user clicks again after switching.

3 Check approval — and close the modal

`approval()` calls `handleClose()` at its top. The form disappears while two wallet prompts are still to come, and a separate progress modal takes over.

4 TX 1: approve — rejection is swallowed

On user rejection the function `return false` s instead of throwing. The caller ignores the return value, so the catch never fires and the flow continues to step 5 with the NFT unapproved.

5 TX 2: createListing, then await the receipt

`await tx.wait(1)` before anything is persisted. This is the correct ordering and it is what buy gets wrong.

6 Receipt status never checked

`receipt.status` is not inspected — same gap as buy. A mined-but-reverted listing proceeds as if it succeeded.

7 Read the listing ID back — may be null

`getListingId` returns `null` on any error and `undefined` if the provider is missing. Either is appended to `FormData` as a string.

8 POST to the backend, response never checked

The result is only read when the domain is premium. For a normal listing the response is discarded and `handleClose()` runs regardless — a rejected save looks identical to a successful one.

02 Is it okay?

Better than buy on the thing that matters most, worse on rejection handling. Side by side:

	BUY	LISTING
Waits for receipt before persisting	no	yes
Checks <code>receipt.status</code>	no	no
Checks the API response	no	premium only
Handles user rejection distinctly	no	partly — broken on approve
Button disabled while submitting	no	no — gated on form validity only
Switches chain inline	no	no
Modal survives the transaction	closes early	closes at approval
Recoverable by the reconciler	yes	no — see below

BUG Rejecting the approval walks you into a guaranteed failure

`approval()` has two exits: it `throw`s on a genuine failure, but `return false`s when the user rejects in their wallet (code 4001). The call site is `await approval(...)` inside a try/catch — the return value is never captured. So on rejection nothing throws, the catch is skipped, and execution continues straight into `createListing` for an NFT the marketplace has no approval on.

The contract requires approval at listing time, so that second transaction reverts. The user declined one wallet prompt, immediately got a second, paid gas on it, and saw a generic "Transaction failed" — when the honest answer was "you cancelled the approval". Worth noting the intent was clearly right: the code has a specific "NFT approval rejected by user." message ready. It just can't be reached.

`DomainListingModal.tsx:158-166 (returns false)` · `:534-552 (return value discarded)`

DATA A listing can go on-chain and never exist off-chain

This is the one place listing is *worse* than buy. If `getListingId` returns null, or the POST fails, the domain is listed on the contract and the backend has no row for it.

The backend guards its side correctly — `create()` opens with `assertNonZeroListingId(...)`, so a bad ID is rejected rather than stored. But that means the request fails, and the frontend never checks the response for non-premium listings, so the seller is shown success.

And the reconciler cannot save this one. It scans for **existing rows** with `blockchainStatus: 'Pending'` — if no row was ever created there is nothing to reconcile. For buy, the same failure is self-healing because the cron independently verifies `NewSale` events against rows it already has. Here the listing is orphaned: live on-chain, invisible in the marketplace, and the seller's NFT stays approved to the contract.

DomainListingModal.tsx:574,595-607 · domain-listing.service.ts:92 · transaction-cron/cron.service.ts:126-130

UX The form closes while two wallet prompts are still pending

`approval()` calls `handleClose()` before sending the approve transaction, so the listing form is gone from step 3 onward. If anything fails afterward, every error path calls `handleClose()` again on an already-closed modal and the user is returned to the domain list with a toast — and **all their form input lost**. Re-listing means retyping the price, description and five keywords.

Same double-submit exposure as buy, too: the submit button is disabled on `!isValid` but not on `isSubmitting`.

DomainListingModal.tsx:148 · :853 · SubmitButton.tsx:18

GOOD What's already right

Persisting only after `tx.wait(1)` is the correct shape, and it's the fix item 8 proposes for buy — so the pattern to copy already exists in this repo. The three-state progress modal gives real feedback across a two-transaction flow. Error messages surface `err.reason` from the revert rather than a flat string. And it checks existing approval first, so re-listing an already-approved NFT correctly skips straight to one transaction.

DomainListingModal.tsx:571 · :533 · :620-624

03 What to change

Four changes, roughly two days, and they fold naturally into item 8's work since both flows need the same treatment.

```

// 1 – one exit contract: approval either succeeds or throws
if (approvedAddr !== marketPlaceAddr) {
  try {
    await approval(tokenIdBN, nftAbi, nft, marketPlaceAddr);
  } catch (e) {
    if (e.code === 4001) return fail('You cancelled the approval.');
```

if (e.code === 4001) return fail('You cancelled the approval.');

return fail('Approval failed. Please try again.');

}

}

// inside approval(): throw on 4001 instead of `return false`

// and drop the handleClose() at the top – keep the form mounted

// 2 – validate the receipt, like the delist flow already does

const receipt = await tx.wait(1);

if (receipt.status !== 1) return fail('Listing reverted on-chain.', tx.hash);

// 3 – the listing ID is required, not optional. Read it from the receipt's

// ListingAdded event rather than a second call that can race the node.

const listingId = readListingIdFromLogs(receipt) ?? await getListingId(tokenIdBN, abi, mark

if (listingId == null || listingId.isZero?()) {

return pendingSync('Listed on-chain. Finishing sync...', tx.hash);

}

// 4 – the save must be checked, for every listing not just premium,

// and must survive a failure rather than being dropped

try {

await setDomainListing(formData).unwrap(); // throws on failure

} catch {

queueForRetry({ formData, txHash: tx.hash }); // persisted locally, retried w/ backoff

return pendingSync('Listed on-chain. Finishing sync...', tx.hash);

}

The orphan case needs a real answer, not just a check. Once step 3 or 4 fails you have a domain listed on-chain that the marketplace doesn't know about, and the reconciler cannot see rows that were never created.

The fix is to make the save **durable rather than earlier**. Keep the insert where it is, after the receipt, but treat it as a job that must complete instead of a fire-and-forget call: retry it with backoff, hold the payload plus `tx.hash` in local storage so a closed tab doesn't lose it, and surface an explicit "listed on-chain, finishing sync" state rather than a success toast. The transaction hash is the durable fact; everything else can be re-derived from it.

Revised. An earlier version of this page recommended writing the row *before* sending the transaction. That was wrong, and `assertNonZeroListingId` is why: `tokenToListing` returns 0 for uninitialised entries, so a pre-send insert has no real listing ID to write. Making it fit would mean relaxing a guard that exists to catch exactly that case — trading a rare orphan for a permanently weaker correctness check on every listing. Not a good trade.

Sequencing

- **Bundle this with item 8.** Both flows need the same four fixes — receipt validation, a submit guard, inline chain switching, and honest rejection handling. Doing them together is roughly a day more than doing buy alone, versus a week of context-reloading later.
- **Bulk listing needs the same pass.** `DomainBulkListingModal.tsx` is 1,464 lines with its own copy of this logic, and it discards the entire five-step wizard on an outside click. It wasn't in the original item 8 scope but shares every fault described here.
- **Item 3 lands on top of this.** Edit-price-without-delist calls `updateListing`, a third write path into the same contract. Worth fixing the shared transaction handling first so it inherits the corrected version rather than adding a fourth copy.